



DRISHTI

SECURITY ARCHITECTURE MODEL

Drishti — Security Architecture & Threat Model

Multi-Layer Threat Modeling, Guardrails, RBAC & Consent Engine



Graph-Powered Attack Paths



Live Watch Agent



Dollar Risk Quantifier



AI Remediation Guardrails



URL Trust Scoring

PLATFORM VERSION

v0.2 Enterprise Edition

RELEASE DATE

August 2026

ARCHITECTURE TIER

Full-Stack Multi-Tenant



RESTRICTED // INTERNAL ARCHITECTURE & TECHNICAL SPECIFICATION // FOR AUTHORIZED USE ONLY

Drishti — Security Model

Reverse-engineered from the implemented product. Every guard, constraint, and check that exists in code — not design intent.

Last updated: 2026-08-17 — verified against source code.

1. Threat model (what the product defends against)

The platform is *purely defensive*. Its internal threat model covers:

Threat	Mitigation	Implementation
Offensive output (model generates exploit code)	GUARDRAIL + output-side marker scan	<code>ai/prompts.py</code> , <code>ai/service.py</code>
** via model**	Scoped context, no in user input	<code>ai/prompts.py</code> (constructions use f-string, not user input)
Tampered asset criticality	Operator-set value is write-through, never agent-overwritten	<code>services/ingest.py: operator criticality rule</code>
Ungated deep-scan	RFC1918-only + explicit consent gate	<code>deepscan/service.py</code>
Unauthorized agent data injection	Hash-verified agent token, org-slug cross-check	<code>api/ingest.py</code> , <code>core/security.py</code>
Token replay after password change	<code>token_version</code> bump invalidates all tokens	<code>services/accounts.py</code>
Login user enumeration	Timing-safe bcrypt with dummy hash for unknown emails	<code>core/security.py</code>
Pre-auth DoS (body flooding)	ASGI streaming body cap (1 MB)	<code>main.py: MaxBodySizeMiddleware</code>
Cross-subnet data collision	Agents only delete within their own observed subnets	<code>services/live.py: observe_devices</code>
Trust-invented device state	MAC is nullable; null MAC = never invented (off-link/L3 only)	<code>models/network_device.py</code>
Prompt exfiltration from context	Context is passed as data, never as a	<code>ai/client.py</code>
Compromised demo data	Demo rows labeled <code>DEMO-ATTACK</code> and cleared before injection	<code>services/live_threats.py</code>
Race condition on ingest	SAVEPOINT + IntegrityError rollback + adopt-or-reread	<code>services/ingest.py</code>

2. Authentication & authorization

User authentication (JWT)

- **Algorithm:** HS256 (`create_access_token`, `create_refresh_token`)
- **Access token:** 15 minutes

- **Refresh token:** 7 days
- **Token payload:** {sub, org_id, type, token_version, iat, exp}
- **Token versioning:** Every password change bumps token_version, invalidating all prior tokens
- **Secret source:** JWT_SECRET env var; **fail-closed** in production (raises if unset, refuses service)
- **Timing-safe login:** Unknown emails still hash against a dummy bcrypt hash — prevents account enumeration via response-time analysis

Agent authentication (hashed token)

- **Token format:** drishti_<urlsafe-base64(24 bytes)>
- **Storage:** SHA256 hash only — plaintext returned once at issuance
- **Lookup:** token_hash = sha256(candidate) equality match
- **Active check:** agent.status == "active" or the call is rejected
- **Org-scope:** Agent's org_id is checked against the request context

Role-based access control

Role	Capabilities
admin	Load sample network, reset org data, rotate agent token
analyst	Read/write findings, assets, AI remediation, live watch
viewer	Read-only access (implied by frontend routing)

Rate limits

Endpoint group	Limit	Burst	Key
Auth (register/login/refresh)	90/min	60	per client IP (+ per email)
Ingest	60/min	20	per agent
AI endpoints	20/min	6	per user
Body size	1 MB hard cap	—	all requests

Stale entries evicted when dict > 10,000 entries (TTL: 1 hour).

3. Data isolation

- Every query that touches user data joins on or filters by org_id
- The agent's org_id is verified at the agent level; the org_slug in the payload is cross-checked against the agent's org, and a mismatch is rejected with ForbiddenError

- The engine loads **only** the current org's data
- JWT payload carries the user's `org_id`; the engine only operates on that org's graph
- **Demo data** is labeled with `_DEMO_LABEL = "DEMO-ATTACK"` and cleared before re-injection

4. Consent gates

Deep-scan consent

PYTHON

```
request.consent is True # 422 if not True
```

- Target must be RFC1918 (private/LAN address space)
- CIDR prefix $\leq$ /22
- Explicit `consent: true` in the request body
- Non-consented / non-private targets are rejected with 422

Subnet scan consent

- `scan_subnet` flag in `AutoScanConfig` — user must explicitly enable it
- Default: scans only the local host (`is_self` device)
- Setting `scan_subnet = True` enables scanning all discovered devices on the subnet

5. AI safety

GUARDRAIL (every)

TEXT

- Only produce DEFENSIVE output
- NEVER produce exploit code, malware, reverse shells, payloads
- If cannot answer defensively $\rightarrow$ `{"refused": true, "reason": "..."}`
- Base answers ONLY on provided context
- Return ONLY valid JSON matching the schema

Output-side defense markers

PYTHON

```
_OFFENSIVE_MARKERS = (  
    "reverse shell", "bind shell", "how to exploit", "weaponize",  
    "establish persistence", "exfiltrate", "attack the target", "ransomware",  
)
```

- Scans: title, summary, script (remediation); headline, narrative, drivers (impact); prediction texts (predict); all text (block)
- If matched: returns `{refused: true, reason: "Request could not be answered defensively."}`
- Deliberately narrower than "exploit"/"payload" — real CVE descriptions use those terms legitimately

Input-side constraints (what's NOT guarded)

- CVE title/description input is **not** blocked on the offensive markers — describing a real vulnerability is defensive context
- Attack path labels (hop labels, CVE titles) are threat data — must always be analyzable
- Asset/neighbor names are inventory data — not subject to input guards

Engine-authoritative dollars

- `POST /api/ai/impact` overwrites the AI's `impact_usd` with the engine-computed value — the AI cannot inflate or deflate financial figures

Mock mode

- `AI MOCK=1` uses canned fixtures or context-specific deterministic templates
- Templates reference the **real** hostname + CVE + port (not fabricated values)
- Hero fixture: PostgreSQL priv-esc CVE → ansible playbook (every other finding gets context-specific template)

6. Defensive posture (honesty model)

What "unknown" looks like everywhere

Service	Honest unknown
Netconfig	<code>{status: "unknown", source: "observed", evidence: " ... "}</code>
Deep scan	<code>{available: false, unavailable_reason: "nmap binary not found"}</code>
URL Trust	<code>{provider: {configured: false}}</code> — absent providers contribute zero weight
Vulnerability ingest	<code>{severity: "unknown", cvss: null}</code> when not provided
Service ingest	Stale open findings not in payload → auto-resolved, not skipped
Connection ingest	Unknown neighbors skipped (not fabricated)

Defensive scopes explicitly documented

- `services/telegram_alerts.py`: "Defensive scope: outbound NOTIFICATION only. No inbound listener, no webhook."
- `services/deepscan/scanner.py`: "Defensive scope: port scan + CVE lookup. No payload generation, no exploit check, no agentic run. Discovery only."
- `agent/drishti_watch.py`: "Agent discloses its mode (ingest/observe/devices) and only runs what was consented to."
- `services/urltrust/`: "Purely defensive: we score and block domains, never attack anything."

7. Input validation

Layer	Validation	Enforcement
Ingest body	Schema (Pydantic): hostname, ip, asset_type, services[], vulnerabilities[], connectivity[]	422 on schema violation
Ingest agent auth	Bearer token hash lookup	401 on invalid/disabled
Ingest org scope	<code>org_slug</code> cross-checked against agent's org	403 on mismatch
Deep scan target	RFC1918 check (ipaddress module)	422 if not private
Deep scan CIDR	Prefix $\leq$ /22	422 if too broad
Deep scan consent	<code>consent is True</code> (not truthy)	422 if not exactly True
Domain observation	Hostname regex + shell-metacharacter stripping	404 "Not a public domain"
Domain cleaning	Strip http(s)://, path, query, www, trailing dot	No injection surface
Body size	1 MB hard cap (streaming)	413 before processing
Hostname validation	<code>^(?=.{1,253}\$)(?!-)[a-z0-9-]{1,63}(?!-)(?:\.(?!-)[a-z0-9-]{1,63}(?!-))+</code>	Regex, rejects shell metacharacters

8. SQL injection safety

- **SQLAlchemy ORM** throughout — no string-concatenated SQL
- **Named parameter binding** in raw queries (e.g., `select(NetworkDevice).where(NetworkDevice.org_id = org_id)`)
- **No raw user strings** in query conditions

9. Secret management

Secret	Source	Stored
JWT signing key	<code>JWT_SECRET</code> env var	In-memory only (used for sign/verify)
User passwords	Plaintext → sha256 → bcrypt	<code>password_hash</code> in DB
Agent tokens	<code>secrets.token_urlsafe(24)</code>	<code>sha256(token_hash)</code> in DB
Groq API key	<code>GROQ_API_KEY</code> env var	In-memory client (never in DB)
NVIDIA API key	<code>NVIDIA_API_KEY</code> env var	In-memory client (never in DB)
Anthropic API key	<code>ANTHROPIC_API_KEY</code> env var	In-memory client (never in DB)
Telegram bot token	<code>TELEGRAM_BOT_TOKEN</code> env var	In-memory only (reads each scan tick)
NVD API key	<code>NVD_API_KEY</code> env var	In-memory only
Vulners key	<code>VULNERS_KEY</code> env var	In-memory only

10. Race condition safety

Pattern	Location	Mechanism
Asset upsert	<code>services/ingest.py</code>	SAVEPOINT + IntegrityError → adopt concurrent row
AutoScanConfig creation	<code>services/autoscan.py</code>	<code>db.begin_nested()</code> + IntegrityError → re-read
Live observation upsert	<code>services/live.py</code>	SAVEPOINT + IntegrityError → adopt concurrent row
Device batch upsert	<code>services/live.py</code>	SAVEPOINT + IntegrityError → adopt concurrent row
Agent token rotation	<code>services/accounts.py</code>	IntegrityError → rollback + ConflictError

11. Security-relevant middleware

- `MaxBodySizeMiddleware` — ASGI streaming body cap (1 MB). Rejects oversized bodies before buffering, preventing pre-auth DoS against `/ingest`.
- CORS** — Allowlist + `chrome-extension://.*` regex. Credentials allowed. The extension's random ID is matched via regex.
- Structured logging** — Every request gets a `request_id` (8-char UUID prefix). Logs include method, path, status, latency_ms.

12. Security gaps and known limitations

Item	Severity	Status
<code>refresh_token</code> endpoint has no rate limit (only auth group is rate-limited)	LOW	By design — refresh tokens are one-time use
No CSRF protection on user endpoints (relies on Bearer tokens, not cookies)	OK	Bearer auth is not susceptible to CSRF
Demo attack injects real-looking threat data	LOW	Clearly labeled; cleared on demand or before re-inject
LLM output could hallucinate CVEs not in context	Mitigated	<code>AI_INSTRUCTIONS.md §2</code> : "Do not invent hosts, CVEs, versions, or numbers" in prompt
No token revocation list (blacklist)	LOW	<code>token_version</code> in user record handles the common case (password change)